LESSON GUIDE

# Technology Plus

## Unit 2: Programming

### 4.3.1 – Identifiers

## Lesson Overview:

In this lesson, students will explore how identifiers, variables, and constants make programs easier to read, understand, and maintain.

**Students will:**

- Define identifiers and explain their purpose in programming.
- Distinguish between variables (changeable values) and constants (fixed values).
- Identify and correct "magic numbers" by replacing them with named constants.
- Rewrite messy code with meaningful identifiers to improve readability.

**Guiding Question:** How do identifiers make code easier to read, write, and understand?

**Suggested Grade Levels:** 8-12

**Technology Needed:** No

**Approximate Time Required:** 60-75 minutes

## Standards:

**CompTIA Tech+ FC0-U71 Objective**

- 4.3 – Explain the purpose and use of programming concepts.
  - **Identifiers**
    - **Variables**
    - **Constants**
  - Arrays
  - Functions
  - Objects
    - Properties
    - Attributes
    - Methods

*This content is based upon work supported by the US Department of Homeland Security's Cybersecurity & Infrastructure Security Agency under the Cybersecurity Education Training and Assistance Program (CETAP).*

# Lesson Background

## Background Information:

In programming, identifiers are the glue that connects humans to code. Without them, programs quickly turn into walls of symbols and numbers that are hard to read and harder to maintain. Introducing identifiers early helps students see that code is not just for machines - it's also written for people.

A variable represents information that can change (like a score or a temperature reading), while a constant represents a value that stays the same (like a maximum number of attempts or the freezing point of water). Clear names make it obvious which is which.

Teachers may notice the phrase "**magic number**" in this lesson. This is programmer slang for a raw number that appears in code without explanation. It isn't a formal certification term, but it's a useful shorthand for helping students understand why replacing those raw values with named constants improves readability. Encourage students to think of constants as a way of giving meaning to otherwise mysterious values.

This lesson also sets up later work with functions and objects. Every function name, object property, or method will also be an identifier. By building a strong foundation here, students will be better prepared to understand those larger programming structures in upcoming lessons.

## Materials Needed:

| | |
|---|---|
| Materials per Class | · Lesson Guide<br>· Lesson Slides 4.3.1 – Identifiers<br>· Activity Slides 4.3.1 – Once Upon an Identifier<br>· Activity 4.3.1 – Once Upon an Identifier Answers (for Teacher Only) |
| Materials per Student | · Student Notes 4.3.1 – Identifiers<br>· Student Handout 4.3.1 – Identifiers<br>· Assessment 4.3.1 – Identifiers (optional) |

# Lesson Background

## Cyber Terms:

| Term | Definition |
|---|---|
| identifier | a label for program elements like variables, constants, functions, and classes |
| variable | a container that can hold changing data |
| constant | a container whose value does not change while a program runs |

# 4.3.1 – Identifiers

**Guiding Question: How do identifiers make code easier to read, write, and understand?**

## Lesson Launch (10 minutes)

**Story Hook: "Who Said That?"**

- Read aloud the story on slides 2-4 (or project and have a student volunteer read).
- The story features four students working on a program, but they are only referred to as "one," "another," "a third," and "the fourth."
- Just like code with poor identifiers, the story is technically correct but frustrating to follow.
- **Ask:** Was that frustrating to follow? Why?

  Students may express that the story wasn't so much frustrating to follow as hard to connect with. When we hear a story, we want to know who the characters are. It helps us set the stage in our mind, helps us imagine what we are hearing/reading.
- **Ask:** What would have made the story clearer?

  Students may say that the story would have been clearer if the characters had names.
- Make the connection to programming:
- Just like people need names in a story, code elements need names that make sense.
- In programming, those names are called identifiers.
- Reveal the Guiding Question on slide 5: How do identifiers make code easier to read, write, and understand?

## Identifying Identifiers (15-20 minutes)

- Distribute Student Handout 4.3.1 – Identifiers and encourage students to fill in the answers during the lesson.

**Identifiers**

- An ***identifier*** labels program elements like variables, constants, functions, and classes. It is the name you give to something in your code.
- Rules vary by language, but usually:
  - Must start with a letter or underscore.
  - Cannot include spaces.
  - Case-sensitive in many languages – `example`, `Example`, and `EXAMPLE` are three different identifiers.
- Identifiers matter for:
  - **Readability:** Good names make code easier to understand (e.g., `userAge` is clearer than `ua`).

- **Maintainability:** Descriptive names help when debugging or revisiting code later.
- **Uniqueness:** An identifier can only refer to one thing at a time within its "scope" (like inside a function). You can't reuse the same name for different things in the same place.

| Allowed (Reassigning a Variable) | Not Allowed (Two Uses of Same Identifier) |
|---|---|
| `def greet_user():`<br>`    user = "Alice"`<br>`    user = "Bob"` | `def greet_user():`<br>`    user = "Alice"`<br>`    def user():   # not allowed`<br>`        print("Hi")` |
| Here, **`user`** is just **one identifier**, but we reassign it. That's fine. | Here we try to use **`user`** as both a variable **and** a function name in the same scope. An identifier can't do double duty, so Python will throw an error to stop us. |

**Variables**

- A ***variable*** is a container that can hold changing data.
- Think of it as a labeled box where the value inside can be replaced.
- Examples:
  - **`score = 10`**
  - **`score = score + 5`** (score is now 15)
- Used for dynamic values: user input, totals, counters, etc.

**Constants**

- A ***constant*** is a container whose value does not change while the program runs.
- Use constants for values that stay the same (like **`PI`** or **`MAX_ATTEMPTS`**).
- **Note:** Constants are often written in ALL CAPS in many languages (e.g., MAX_SCORE). It's not a rule, but it's a convention that helps distinguish constants at a glance.
- Benefits:
  - Improves readability (**`MAX_SCORE`** instead of just 100).
  - Avoids "magic numbers" in code.
  - **Note:** A "magic number" is just a raw number that shows up in code without explanation. It "magically" works, but the purpose isn't clear to someone reading (or even to *future you* coming back later).

| "Magic Number" | Clearly Identified Constant |
|---|---|
| `if score > 100:` | `MAX_SCORE = 100`<br>`if score > MAX_SCORE:`<br>`  print("Winner!")` |
| Why 100? Is it:<br>• the maximum score?<br>• a passing grade?<br>• some random test value?<br>We don't know. That's the "magic." | Now it's clear – the program is comparing to the maximum score, not just some mysterious 100. |

- **Note:** Identifiers don't stop at variables and constants - they'll also name functions and objects later in this unit.

## Activity: Once Upon an Identifier (30-35 minutes)

- Display Activity Slides 4.3.1 – Once Upon an Identifier and direct students to the table for Once Upon an Identifier on the student handout.
- Decide whether you want students to work alone, in pairs, or in groups and facilitate accordingly.
- Use slide 2 to give students directions for how they will complete this activity.
- For each row of the table, students will:
  - Look at the messy code in **Column 1**. Decide what story the code could tell if the identifiers had better names.
  - Rewrite the code in **Column 2** with clearer identifiers and replace any "magic numbers" with constants.
  - Leave **Column 3** blank for now. Later, you'll swap papers with another group. You'll read their fixed code and write a short story in **Column 3** about what their code does.
- While students will invent their own identifiers and stories, sample answers are provided as guidance. These show what's wrong with each snippet, a possible fix, and one story interpretation.
- Slides 3-12 provide the messy code snippets for share-out.
- During the share-out:
  - Students/groups present the stories they wrote about the fixed code they received.
  - The group who originally fixed the code confirms or corrects the interpretation based on their own vision.

## Putting It All Together (5-10 minutes)

- **Ask:** Why do programmers use identifiers instead of single letters like x or y?

  Sample Answer: Because identifiers make code clearer to humans – they describe the purpose of the value, which helps with readability and debugging.

- **Ask:** What's the difference between a variable and a constant?

  Sample Answer: A variable can change during the program, like studentScore, while a constant stays the same, like PASSING_SCORE.

- **Ask:** What is a "magic number," and how do constants help avoid them?

  Sample Answer: A magic number is a raw, unexplained value in code. Using a constant like FREEZING_POINT = 32 replaces the mystery number with something descriptive.

- **Ask:** How did better identifiers help you tell the "story" of your code in today's activity?

  Sample Answer: With good names, we could explain what the program was doing (like adding cookies, checking login attempts, or tracking grades) without guessing.

- Use slides 14-15 to revisit the story from the lesson launch, but this time with clear identifiers.